

BÁO CÁO BÀI TẬP: ỨNG DỤNG AI TẠO SINH TRONG SÁNG TẠO NỘI DUNG SỐ

- Họ và tên sinh viên: Lê Phương Nam
- MSSV: 2502333
- Chủ đề dự án: Bài viết chuyên đề (Blog Post/Article): "*Giải ngố Thuật toán Dijkstra: Từ bản đồ Google Maps đến tư duy lập trình C++*".

I. MÔ TẢ DỰ ÁN

- Sản phẩm:** Một bài viết chuyên sâu nhưng dễ hiểu (khoảng 800 - 1000 chữ) giải thích cơ chế hoạt động của thuật toán tìm đường đi ngắn nhất Dijkstra. Bài viết được thiết kế dưới dạng một bài blog công nghệ, có kèm hình ảnh minh họa và mã nguồn tham khảo.
- Mục tiêu:** Giúp các bạn sinh viên năm nhất ngành Trí tuệ nhân tạo hoặc Công nghệ thông tin dễ dàng nắm bắt logic cốt lõi của đồ thị và quy hoạch động trước khi bước vào thực hành viết code C++.

II. QUÁ TRÌNH SỬ DỤNG AI

1. Công cụ AI tạo văn bản: ChatGPT (GPT-4) & Google Gemini

- Mục đích:** Xây dựng dàn ý (outline), tạo bản nháp nội dung và giải thích các khái niệm phức tạp bằng ngôn ngữ đời thường.
- Prompt 1 (Dàn ý - trên ChatGPT):** "*Đóng vai một blogger công nghệ có chuyên môn sâu về Khoa học máy tính. Hãy lên cho tôi một dàn ý chi tiết cho bài viết 'Giải ngố Thuật toán Dijkstra'. Đối tượng đọc là sinh viên năm nhất. Yêu cầu có phần liên hệ thực tế và phần giải thích cách cài đặt cơ bản.*"
 - Kết quả:** Nhận được dàn ý 4 phần: Mở đầu thu hút (ví dụ Google Maps), Nguyên lý hoạt động (các bước chọn đỉnh, cập nhật trọng số), Lưu ý về hiệu suất, và Kết luận.

Giải ngố Thuật toán Dijkstra

1. Mở đầu thu hút (ví dụ Google Maps)

- Bạn đã bao giờ thắc mắc:
 - Tại sao Google Maps luôn tìm được đường đi nhanh nhất gần như ngay lập tức?
- Thực tế:
 - Đây là một bài toán kinh điển trong Khoa học máy tính: **tim đường đi ngắn nhất**
- Giới thiệu:
 - Thuật toán Dijkstra do Edsger W. Dijkstra đề xuất
 - Là nền tảng của nhiều hệ thống hiện đại
- Mục tiêu bài viết:
 - Giải thích dễ hiểu cho sinh viên năm nhất
 - Hiểu được cả **cách hoạt động** và **cách cài đặt cơ bản**

2. Nguyên lý hoạt động (các bước chọn đỉnh, cập nhật trọng số)

2.1 Mô hình bài toán

- Đồ thị:
 - Đỉnh = địa điểm
 - Cạnh = đường đi
 - Trọng số = khoảng cách/thời gian
- Mục tiêu:
 - Tìm đường đi ngắn nhất từ một điểm đến các điểm khác

2.2 Ý tưởng chính

- Luôn chọn:
 - Đỉnh có khoảng cách nhỏ nhất  ý lý
- Sau đó:
 - Cập nhật khoảng cách đến các đỉnh kề

3. Lưu ý về hiệu suất

3.1 Độ phức tạp

- Khi dùng priority queue:
 - $O((V + E) \log V)$

3.2 Cấu trúc dữ liệu quan trọng

- Adjacency list:
 - Lưu đồ thị hiệu quả
- Priority queue:
 - Luôn lấy được đỉnh gần nhất nhanh nhất



4. Kết luận

- Dijkstra là:
 - Một trong những thuật toán nền tảng quan trọng nhất
- Ứng dụng:
 - Từ bản đồ, game đến mạng máy tính
- Với sinh viên năm nhất:
 - Nên hiểu:
 - ý tưởng
 - cách hoạt động
 - cách code cơ bản
- Gợi ý:
 - Tự implement lại từ đầu để nắm chắc

- Prompt 2 (Viết nháp - trên Gemini):** *"Dựa trên dàn ý phần 2, hãy viết một đoạn khoảng 300 chữ giải thích bước 'Cập nhật trọng số' (Relaxation) của thuật toán Dijkstra. Hãy dùng giọng văn gần gũi, xưng 'mình' và gọi người đọc là 'bạn'."*
 - Kết quả:** Gemini sinh ra một đoạn giải thích khá mượt mà, tuy nhiên phần diễn giải về thuật toán gặp một chút rập khuôn và hơi máy móc.

Giải thích bước Cập nhật trọng số Dijkstra

Hiện tiến trình tư duy

Tiếp nối dàn ý ở phần trước, bây giờ mình sẽ cùng bạn giải phẫu "trái tim" của thuật toán Dijkstra: bước **Cập nhật trọng số** (trong tiếng Anh gọi là *Relaxation*).

Thú thật, lần đầu nghe thuật ngữ "Relaxation" (thư giãn), mình đã nghĩ bước này chắc nhàn hạ lắm. Nhưng thực chất, đây lại là lúc thuật toán phải liên tục "cân não" để tối ưu hóa đường đi.

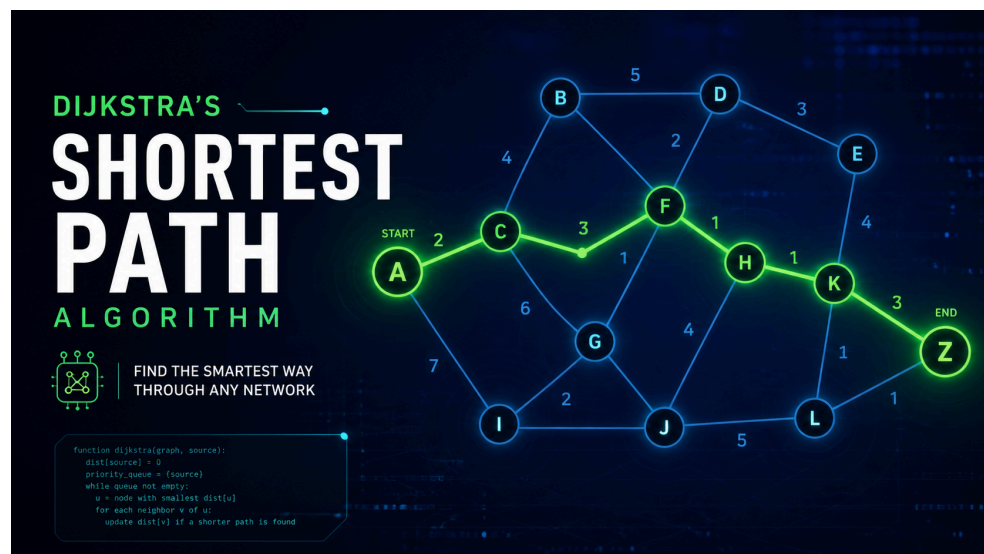
Để mình lấy một ví dụ thực tế cho bạn dễ hình dung nhé. Giả sử bạn đang tìm con đường ngắn nhất từ KTX đến giảng đường. Ban đầu, dựa trên những gì đã biết, bạn ước tính đi theo lộ trình cũ sẽ mất 15 phút. Tuy nhiên, khi bạn đi đến một ngã tư mới (chúng ta gọi nó là đỉnh u), bạn chợt nhận ra: "Khoan đã, nếu từ ngã tư u này mình rẽ sang con hẻm nhỏ nối thẳng tới giảng đường (đỉnh v), thì tổng thời gian từ KTX đến đây cộng thêm đoạn hẻm đó chỉ mất có 10 phút!". Rõ ràng, bạn sẽ gạt bỏ con số 15 phút cũ kỹ kia đi và tự động "cập nhật" trong đầu lộ trình 10 phút mới mẻ và tiết kiệm thời gian hơn.

Dưới góc độ logic của đồ thị, thao tác này diễn ra cực kỳ dứt khoát. Khi mình đang chốt khoảng cách ở đỉnh u và xét một đỉnh kế v , mình sẽ thực hiện một phép thử đơn giản. Mình đặt lên bàn cân so sánh:

- Khoảng cách ngắn nhất hiện tại đang được lưu từ đỉnh nguồn đến v (tức là $\text{dist}[v]$).
- Chỉ phí nếu quyết định đi đường vòng qua u (tức là $\text{dist}[u]$ cộng với trọng số cạnh nối u và v).

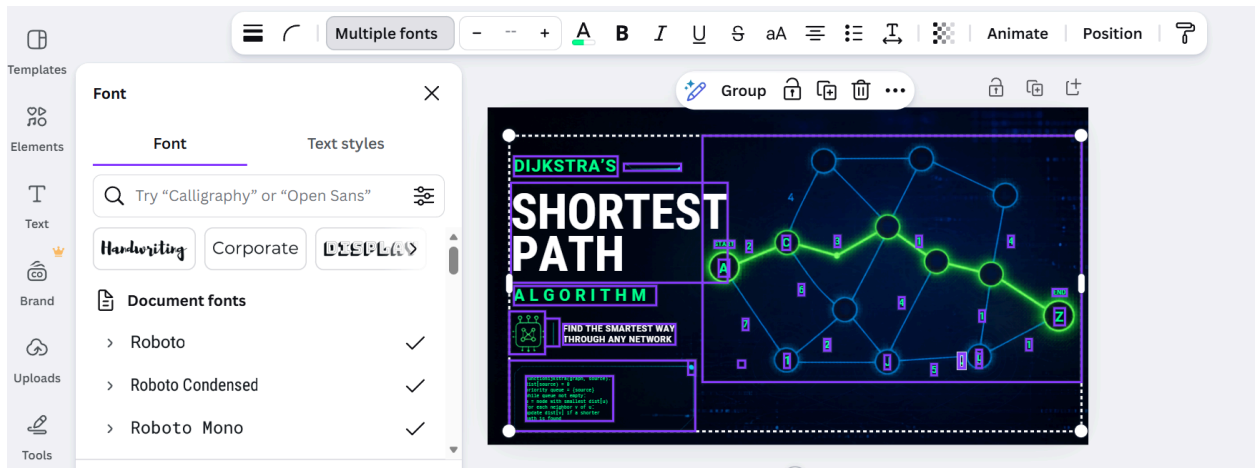
2. Công cụ AI tạo hình ảnh: DALL-E 3

- **Mục đích:** Tạo ảnh bìa (Thumbnail/Cover) hấp dẫn cho bài viết blog và một hình minh họa các nút (node) trong đồ thị.
- **Prompt:** *"A visually striking blog cover image representing Dijkstra's shortest path algorithm. Show a glowing path connecting two nodes in a complex tech-style network graph. Dark blue background with neon green and cyan accents. 16:9 aspect ratio, high resolution, minimalist."*
 - **Kết quả:** DALL-E 3 tạo ra bức ảnh mạng lưới đồ thị mang đậm phong cách không gian mạng (cyberpunk), làm nổi bật đường đi ngắn nhất rất trực quan.



3. Công cụ AI hỗ trợ thiết kế: Canva AI (Magic Design)

- **Mục đích:** Thiết kế lại banner bài viết và chèn typography (chữ nghệ thuật) sao cho chuẩn form của một bài viết trên các nền tảng như Medium hay Spiderum.
- **Thao tác:** Tải ảnh từ DALL-E 3 lên Canva, dùng công cụ "Magic Grab" để tách nền các điểm sáng, sau đó dùng AI đề xuất font chữ mang phong cách Coding (như Fira Code hoặc Roboto Mono) để ghép tiêu đề bài viết vào ảnh bìa.



III. CHỈNH SỬA VÀ TÍCH HỢP (YẾU TỐ CON NGƯỜI)

AI là một người trợ lý viết lách rất nhanh, nhưng thiếu đi sự sắc sảo trong các tiểu tiết học thuật. Dưới đây là những đóng góp sáng tạo và sự điều chỉnh của em:

- **Sửa lỗi logic thuật toán:** Trong quá trình Gemini viết nháp phần hướng dẫn chạy thuật toán, AI đã đưa ra một bước đánh giá sai về cách chọn đỉnh tiếp theo trong hàng đợi ưu tiên (Priority Queue). Nhận thấy lỗi logic này khá nghiêm trọng, em đã trực tiếp xóa bỏ đoạn diễn giải đó và tự tay viết lại quy trình cập nhật khoảng cách (khoảng cách mới = khoảng cách đến đỉnh hiện tại + trọng số cạnh), đảm bảo tính chính xác tuyệt đối.
- **Bổ sung mã nguồn thực tế:** AI có xu hướng viết giả mã (pseudo-code) chung chung. Để bài viết thực sự có giá trị thực chiến, em đã tự code và chèn một đoạn mã C++ tối ưu sử dụng `priority_queue` và danh sách kề (vector pair) vào cuối bài viết, có kèm bình luận (comment) giải thích cách cấp phát bộ nhớ.
- **Định hình văn phong:** Em đã sửa lại những câu chuyển đoạn sáo rỗng do ChatGPT viết (ví dụ: "Trong thế giới công nghệ số ngày nay...") bằng những câu mở đầu mang tính trực diện và cá nhân hơn.

IV. PHÂN TÍCH VAI TRÒ CỦA AI TRONG QUÁ TRÌNH SÁNG TẠO

1. Điểm mạnh và Điểm hạn chế:

- **Điểm mạnh:** AI giải quyết triệt để hội chứng "sợ trang giấy trắng". Việc có ngay một dàn ý rành mạch và các đoạn nháp thô chỉ trong 5 phút giúp em có đà để viết tiếp. Khả năng sinh ảnh của DALL-E 3 thay thế hoàn toàn việc phải đi tìm ảnh stock vi phạm bản quyền trên mạng.
- **Hạn chế:** AI sinh văn bản thường thiếu chiều sâu chuyên môn khi đi vào các bài toán hóc búa. Các câu văn thường thiếu "độ nảy" và mang nặng cảm giác dịch thuật. Nếu không có nền tảng kiến thức vững, người viết rất dễ bị AI dẫn dắt sai lệch về mặt thuật toán.

2. Thay đổi quy trình làm việc: Thay vì cặm cụi tự gõ từng chữ từ đầu đến cuối, quy trình của em chuyển sang hướng **Đạo diễn nội dung**: *Lên ý tưởng -> Giao việc cho AI (Prompting) -> Giám định nội dung -> Cắt gọt, thêm thắt chất xám cá nhân -> Đóng gói.*

3. Vấn đề đạo đức: Tính toàn vẹn của thông tin là yếu tố quan trọng nhất. Nếu một sinh viên khác đọc bài viết này và áp dụng sai thuật toán vào bài thi chỉ vì lỗi "ảo giác" của AI, người viết phải chịu hoàn toàn trách nhiệm. Do đó, việc kiểm duyệt chéo bằng kiến thức học thuật là bắt buộc. Ngoài ra, việc ghi chú rõ "Bài viết có sử dụng AI trong khâu dàn ý và hình ảnh" ở cuối bài là cần thiết để giữ gìn sự minh bạch.

V. SẢN PHẨM CUỐI CÙNG (FINAL PRODUCT - TRÍCH ĐOẠN BÀI VIẾT)

Tiêu đề: Giải ngố Thuật toán Dijkstra: Từ bản đồ Google Maps đến tư duy lập trình C++

Bạn đã bao giờ tự hỏi làm thế nào Google Maps có thể vạch ra tuyến đường nhanh nhất từ nhà bạn đến trường trong chưa đầy 1 giây, giữa hàng vạn con phố ngằng ngoèo? Câu trả lời nằm ở một thuật toán kinh điển ra đời từ năm 1956: Thuật toán Dijkstra.

1. Dijkstra hoạt động như thế nào? (Nói không với thuật ngữ lằng nhằng) Hãy tưởng tượng bạn là một người giao hàng. Bạn đứng ở điểm A và cần đến điểm Z. Các ngã tư là các "đỉnh" (node), và các con đường nối ngã tư là các "cạnh" (edge). Độ dài hoặc thời gian đi qua con đường đó chính là "trọng số" (weight).

Dijkstra không chạy lung tung. Nó có một chiến thuật cực kỳ chắc chắn:

- Đầu tiên, nó ghi nhận khoảng cách đến tất cả các ngã tư là "vô cực" (vì chưa đi thử nên chưa biết), riêng điểm bạn đang đứng là 0.
- Tiếp theo, nó luôn chọn đi đến ngã tư có khoảng cách *ngắn nhất* mà nó biết.
- **Trái tim của thuật toán:** Khi đến một ngã tư mới, nó sẽ nhìn ra xung quanh. Nếu phát hiện ra rằng: *Đường từ nhà -> ngã tư hiện tại -> ngã tư tiếp theo* NGẮN HƠN con đường cũ đã ghi sổ, nó sẽ gạch bỏ số cũ và cập nhật ngay khoảng cách mới!

2. Ứng dụng vào thực chiến Thuật toán là lý thuyết, nhưng để nó thực sự chạy nhanh, chúng ta phải biết cách tổ chức dữ liệu. Khi cài đặt bằng C++, thay vì phải quét toàn bộ các đỉnh để tìm khoảng cách nhỏ nhất (rất tốn thời gian), những lập trình viên kinh nghiệm sẽ sử dụng

`priority_queue` (hàng đợi ưu tiên) để các đỉnh có khoảng cách ngắn nhất tự động nổi lên trên cùng...